

高级语言程序设计大作业报告

姓名： 王韩淼淼 学号： 2511223

班级： 信息安全、法学双学位班

2026年5月

目录

1	作业题目	1
2	开发软件	1
3	课题要求	1
4	主要流程	1
4.1	整体流程	1
4.2	核心算法设计	2
4.2.1	地图随机生成算法 (DFS + BFS)	2
4.2.2	回合制战斗模拟	2
4.2.3	网络粘包处理协议	2
4.3	模块划分	2
5	单元测试与网络通信	4
5.1	本地单元测试	4
5.2	网络通信测试	4
6	开发难点与解决	4
6.1	迷雾系统与视野更新	4
6.2	TCP粘包处理	5
6.3	一方移动在另一方界面实时同步	5
6.4	音效播放时长控制	5
6.5	对象的生命周期管理	5
6.6	AI工具的利用	5
6.7	AI工具的利用	6
7	收获与总结	6
7.1	对Qt框架的深刻理解	6
7.2	项目版本管理 (Git)	6
7.3	设计模式的实践	6
7.4	不足与改进	6

1 作业题目

《地牢勇士——基于Qt的Roguelike探险与多人对战游戏》

本项目实现了一款具有Roguelike元素的2D地牢探险游戏。玩家控制角色在随机生成的地牢中探险，通过战斗提升属性，收集道具，最终抵达底层。项目创新点在于结合了局域网多人对战功能，两名玩家可以在各自探索的地牢中积累属性，并通过“遭遇战”模式进行实时1v1对决。

2 开发软件

- IDE: Qt Creator 6.0
- 辅助工具: Git (代码版本管理)、B站 (视频演示)

3 课题要求

- (1) 图形化界面: 使用Qt框架, 实现无边框窗口、动态迷雾、动画贴图等效果。
- (2) 面向对象: 运用继承 (基类工厂)、多态 (不同地块类型) 和封装 (各功能模块)。
- (3) 网络编程: 基于TCP协议实现自定义应用层协议, 支持多客户端登录、房间管理和实时同步。
- (4) 复杂度与创新: 实现了BFS/DFS地图连通性算法、回合制战斗模拟、粘包处理机制等。
- (5) 项目报告与展示: 遵循Git提交规范, 录制B站讲解视频。

4 主要流程

4.1 整体流程

游戏分为单机模式和网络对战模式两个核心玩法。

- 单机模式: 登录 → 选择角色 → 进入1-7层地牢 → 击败怪物/拾取道具 → 提升属性 → 走出地牢。
- 网络对战模式: 登录 → 创建/加入房间 → 双方准备 → 进入独立PK地图 → 80秒发育倒计时 → 遭遇战斗 → 一人死亡。

4.2 核心算法设计

4.2.1 地图随机生成算法（DFS + BFS）

为了保证每层地图既有随机性又有解，采用了**模板+随机填充**策略：

1. **DFS寻路**：使用深度优先搜索在预设的“墙”结构中随机打通一条从起点到终点的主干道。
2. **BFS验证**：生成怪物和物品后，使用广度优先搜索验证起点到终点是否连通。若不连通，则强行清空路径上的障碍物，确保玩家不会卡死。

4.2.2 回合制战斗模拟

战斗采用经典RPG公式：

$$\text{玩家伤害} = \text{角色攻击} - \text{怪物防御} \quad (1)$$

$$\text{怪物伤害} = \text{怪物攻击} - \text{角色防御} \quad (2)$$

战斗流程：玩家先手攻击，怪物反击，循环直到一方生命值归零。胜负判定包括胜利、失败、无法破防三种情况。

4.2.3 网络粘包处理协议

由于TCP是流式协议，自定义了以短横线（-）作为结束标志的协议。接收缓冲区中的数据按结束标志分割，完整消息立即处理，不完整的数据保留等待下次接收。

4.3 模块划分

本项目共包含14个源文件，模块划分如下：

表 1: 项目模块划分（完整版）

序号	模块文件	主要功能	关键技术点
1	dungeon.cpp	地牢核心逻辑	迷雾系统、地图生成、战斗结算、事件分发、网络对战
2	mainwindow.cpp	主窗口与布局	无边框窗口、网格布局、键盘事件转发、鼠标拖动
3	interface.cpp	启动界面与主菜单	加载动画、进度条、模式选择
4	chatroom.cpp	聊天室与网络通信	TCP粘包处理、消息序列化、PK同步
5	rooms.cpp	房间管理	房间列表、创建/加入/离开、准备/开始机制
6	sign.cpp	登录注册窗口	TCP连接、注册/登录请求、会话管理
7	menu.cpp	游戏菜单	暂停菜单、继续/重开/返回/帮助/音乐开关
8	sound.cpp	音效管理	背景音乐循环、短音效时长控制
9	dungeonsoundmanager.cpp	地牢音效管理器	播放列表管理、自动切歌、音量控制
10	storewidget.cpp	商店界面	金币购买属性、价格递增、边框动画
11	help.cpp	帮助窗口	按键说明展示、返回按钮
12	factory.cpp	UI控件工厂	统一创建控件、默认样式、多参数重载
13	main.cpp	程序入口	QApplication、多语言翻译、显示主窗口
14	server_main.cpp	游戏服务器	TCP监听、用户认证、房间管理、消息转发

5 单元测试与网络通信

5.1 本地单元测试

- **战斗平衡测试：** 测试不同攻击力差值下战斗函数的返回值正确性。
- **地图连通性测试：** 随机生成100张地图，验证BFS是否总能找到起点到终点的路径。

5.2 网络通信测试

使用两个客户端实例模拟局域网对战。

表 2: 网络功能测试用例

测试场景	预期结果	实际结果
客户端A创建房间	房间列表显示房间A	通过
客户端B加入房间	双方界面显示对方头像	通过
房主未准备时点开始	按钮无效或提示	通过
非房主点击准备	房主端按钮变为可开始	通过
PK移动同步	双方屏幕显示对手移动轨迹	通过
服务器意外断开	客户端提示重连	待完善

6 开发难点与解决

6.1 迷雾系统与视野更新

问题： 玩家移动时，需要动态更新周围的可见区域，且每个格子只能生效一次，避免反复清除迷雾造成视野异常。

解决： 维护一个三维数组 `FogArr[floor][x][y]` 记录每个格子的雾浓度（0-15，数值越大雾越浓）。玩家首次踏入某格时，调用 `updateFogArea` 函数，利用 `haveVisited` 数组确保每个格子只生效一次。同时向周围四个方向传播不同的雾浓度递减值（上方减12、下方减3、左方减5、右方减10），模拟地牢中灯光从头顶照下来的不对称视野效果。当雾浓度达到8或以上时，该格子完全可见。

6.2 TCP粘包处理

问题：网络通信中，TCP是流式协议，多条消息可能粘连在一起发送（粘包），或一条消息被拆成多个片段发送（拆包）。如果不做处理，会导致消息解析错误，例如PK请求和聊天消息混在一起无法区分。

解决：设计了自定义应用层协议，每条消息以短横线（-）作为结束标志。接收端维护一个接收缓冲区，每次收到数据后先追加到缓冲区，再按结束标志分割。完整消息立即调用 `handleServerMessage` 处理，不完整的数据保留在缓冲区中等待下次数据到达。实测该方案在局域网环境下稳定运行，未出现消息错乱。

6.3 一方移动在另一方界面实时同步

问题：联网对战中，玩家A移动后，玩家B的屏幕上需要实时显示A的位置变化。如果直接发送每个坐标，网络开销大且容易因延迟导致卡顿。

解决：采用客户端驱动 + 服务器转发模式。玩家按下移动键时，客户端立即在本端执行移动（保证操作流畅），同时将移动方向和最终坐标通过服务器转发给对手。对手收到位置更新消息后，通过定时器刷新UI，将对手图标移动到新位置，并根据移动方向切换不同帧的行走动画。该方案兼顾了流畅性和同步精度。

6.4 音效播放时长控制

问题：游戏中有多种短音效（金币声、攻击声、死亡声等），不同音效需要不同的播放时长（普通音效约1秒、短促音效约0.4秒、长音效约3秒）。

解决：在 `DungeonSoundManager` 类中实现播放列表管理。播放音效时，根据音效索引设置不同的定时器停止时间：索引3（踩踏声）设置400ms后停止，索引5（死亡声）设置3000ms后停止，其他音效设置1000ms后停止。这样可以避免音效重叠播放造成的混乱。

6.5 对象的生命周期管理

问题：在无边框窗口模式下，动态创建的控件如果不指定父对象，容易造成内存泄漏。此外，三维地图数组也需要手动释放。

解决：利用Qt的对象树机制，在构造函数中为所有控件指定当前窗口（`this`）作为父对象，Qt会自动管理其生命周期。对于三维地图数组 `map`，在 `Dungeon` 类的析构函数中按三维数组的逆序逐层释放内存，防止内存泄漏。

6.6 AI工具の利用

- 工具：GitHub Copilot、GPT-4

- **用途：** 辅助生成BFS/DFS算法样板代码、调试TCP粘包处理逻辑、生成正则表达式用于解析服务器消息、编写部分UI样式表、润色实验报告。

6.7 AI工具の利用

- **工具：** GitHub Copilot、ddepseek
- **用途：** 调试TCP粘包处理、报告润色。

7 收获与总结

7.1 对Qt框架的深刻理解

掌握了网格视图架构、定时器多线程处理、多媒体播放、TCP套接字编程，理解了事件循环的重要性。

7.2 项目版本管理（Git）

实现了不少于3次阶段性提交：初版框架 → 单机逻辑 → 网络模块 → UI美化。

7.3 设计模式的实践

使用了工厂模式创建UI控件、单例模式管理音效、观察者模式处理信号槽通信。

7.4 不足与改进

- **服务器稳定性：** 当前服务器未做心跳包检测，若客户端异常断开，服务器需较长时间才能感知并清理房间。后续可增加心跳机制，定时检测客户端存活状态。
- **网络延迟问题：** 局域网环境下对战较为流畅，但在网络状况较差时（如WiFi信号不稳定或跨网络场景），存在明显的操作延迟和位置同步滞后现象。后续可考虑使用帧同步或状态同步等更成熟的网络同步方案，并增加延迟补偿机制。
- **跨平台性：** 当前音效路径使用了 Qt 资源文件（qrc），在 Linux 下编译时部分音频格式需要转换。后续可统一使用跨平台兼容性更好的音频格式（如 WAV），或根据不同平台动态加载对应格式的音效文件。
- **界面体验：** 部分 UI 交互反馈不够细腻，如按钮悬停效果、战斗动画过渡等。后续可增加更多视觉特效和音效反馈，提升玩家沉浸感。

结语

本次大作业锻炼了C++编码能力，体验了从零到一完成可玩游戏的成就感。

附件：项目信息

- github仓库地址: <https://github.com/gaxaly814/dungeon>
- B站讲解视频地址: <https://www.bilibili.com/video/BV1Zv5v6KEvX>
- 视频标题: 【南开大学26C++】一个人刷地牢，两个人决斗！单机联机两不误